

that Google runs billions of containers per week to process requests from some of its largest Web properties like Gmail, Google Docs and others. So once Docker streamlined the containerisation of apps via its container format and tools, Google thought it fit to open its expertise acquired over the last decade in running distributed containerised workloads via a modified version of Borg that would provide a battle-tested orchestration system. Thus Kubernetes was born and has been available as open source for a while.

The Kubernetes ecosystem

The Kubernetes project is available at the GitHub URL <https://github.com/kubernetes/kubernetes> and has been one of the most active projects over the last 12-18 months. Major infrastructure providers and Linux vendors have supported this project and have been contributing to it. A significant portion of the code being contributed to Kubernetes is from major players like Red Hat, Citrix, IBM and others.

In addition to the core contributors, an entire ecosystem of vendors is evolving around Kubernetes. These vendors are providing a higher layer abstraction around it, offering tools around visualising or managing your Kubernetes clusters, CI/CD workflows and more. This is probably the right time to get into this space since, currently, it is being actively worked on, and it is widely accepted that containers and orchestration tools will be the way we deploy our applications, moving forward.

The need for Kubernetes

So far, we covered some brief points about why we need Kubernetes, its history and its current momentum in the industry. It is important to highlight the technical reasons we need an orchestration layer for our container applications, especially when we are talking of large clusters running hundreds and thousands of containers.

Let us look at a few key requirements when you are dealing with distributed workloads.

Auto-scaling: As the load on your application increases or decreases, you require a system that takes care of launching more containers in the pool or bringing down the containers.

Health checks and readiness: In a cluster, it is important to know whether the containers are healthy or not, and whether they are ready to take the traffic routed to them or not. This ensures that the system can start up other containers, mark the unresponsive ones as bad and manage the traffic across healthy containers.

Portability: Docker has gained widespread acceptance due to the fact that it actually enables portability. This means that the container runtime is portable across infrastructure and operating systems, and the tools are simple, lean and available on not only the high-end machines but also on our laptops.

Rolling updates: All software undergoes changes due to newer features or fixes. In a system that is running with hundreds of containers, it is essential to have a smooth mechanism to roll out updates to the containers without

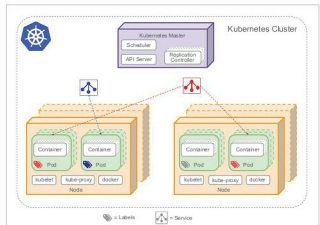


Figure 1: Kubernetes cluster

bringing the whole system down. This ensures the availability of the application.

Extensibility: To encourage a healthy ecosystem, it is important for the system to be modular in nature, with extensibility provided via plugins that can be written by software vendors and individual developers.

Kubernetes meets the above requirements, all of which are expected from a container orchestration layer.

The building blocks

To understand Kubernetes a bit better, it is important to get familiar with its terminology. This helps when you start to download and explore Kubernetes.

Pod: The first thing to understand is a pod, which is the smallest deployable unit. Think of it as a collection of containers that logically form an application. For example, if you are writing a Web application running in a Web server that is supported by a MySQL database at the backend, you can think of the pod as running your Web application with a couple of containers, one that runs the Web application and the other that runs your database. All the containers in the pod can share the volumes, and a pod will be scheduled on one or more nodes.

Node: A node is a physical machine or virtual machine running Kubernetes. The pods will be scheduled to run on one or more nodes. Each of the nodes will have the Docker engine that is responsible for eventually running the containers, though Kubernetes also supports another runtime—rkt from CoreOS. It will also have a kubelet, which takes the commands from the master. The kube-proxy is a proxy that runs on each node and helps enable the services, which is mentioned later in the Service section.

Cluster: A Kubernetes cluster includes all your physical machines, virtual machines and the infrastructure on which you are going to run your applications. These clusters could be co-located in a single data centre, across data centres or even regions. The goal of Kubernetes is to run seamlessly across any of the physical regions in a transparent manner for the developers.

Replication controller: Earlier, we spoke about the need to bring containers up and down. More specifically, since pods are